

Red Black Tree

A Balanced Binary Tree

S.M. Kausar Parvej
Id: 2005076

Sagor Chanda
Id: 2005071

Souvik Mandal
Id: 2005069

March 9, 2024



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Contents

1	Introduction	3
2	Red Black Tree	3
3	Properties	4
4	How Red Black Tree is Height Balanced	5
5	Operations	6
5.1	Searching	6
5.2	Insertion	9
5.3	Deletion	15
6	Practical Applications	17
7	Conclusion	18

1 Introduction

Trees are well-known as a non-linear data structure[4]. They don't store data in a linear way. They organize data hierarchically. One specific type of a tree is Binary Search Tree. The binary search tree data structure supports many dynamic-set operations, including SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERTION, and DELETION.

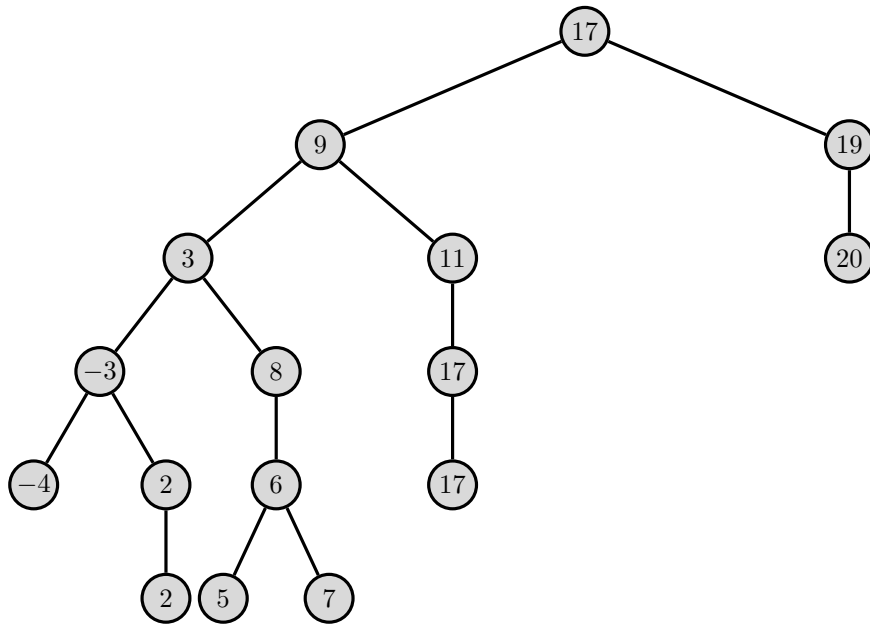


Figure 1: Example of a Tree

But these basic operations on a binary search tree take time proportional to the height of the tree. For a complete binary tree with n nodes, such operations run in $\mathcal{O}(\log n)$ worst-case time. If the tree is a linear chain of n nodes, however, the same operations take $\mathcal{O}(n)$ worst-case time. So we need trees which can balance its height by itself. Red Black Tree is a kind of such Height Balanced Binary Search Trees.

2 Red Black Tree

A red-black tree is a kind of self-balancing binary search tree in computer science. Each node of the binary tree has an extra bit, and that bit is often interpreted as the color (red or black) of the node. By constraining the node colors on any simple path from the root to a leaf, red-black trees ensure that no such path is more than twice as long as any other. The tree is thus ap-

proximately height balanced. Each node of the tree now contains the attributes *color*, *key*, *left*, *right*, and *p*. If a child or the parent of a node does not exist, the corresponding pointer attribute of the node contains the value **NIL**. These **NILs** are regarded as being pointers to leaves (external nodes) of the binary search tree and the normal, key-bearing nodes as being internal nodes of the tree.

3 Properties

The properties[3] of a Red Black Tree is given below:

1. **Color Property:** Each node is either red or black.
2. **Root Property:** The root is black
3. **External Property:** Every external node is black
4. **Internal Property:** Both children of a red node are black.
5. **Depth Property:** For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

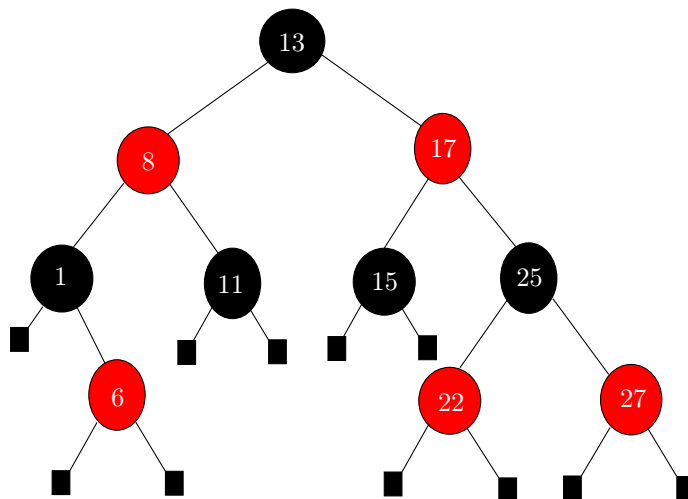


Figure 2: Properties of a Red Black Tree

The number of black nodes from the root to a node is the node's black depth; the uniform number of black nodes in all paths from root to the leaves is called the black-height of the red-black tree.

These constraints enforce a critical property of red-black trees: the path from the root to the farthest leaf is no more than twice as long as the path from the root to the nearest leaf. The result is that the tree is roughly height-balanced. Since operations such as inserting, deleting, and finding values require

worst-case time proportional to the height of the tree, this theoretical upper bound on the height allows red-black trees to be efficient in the worst case, unlike ordinary binary search trees.

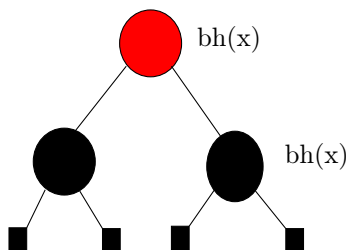
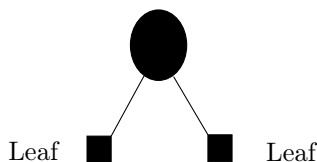
To see why this is guaranteed, it suffices to consider the effect of properties 4 and 5 together. For a red-black tree T , let B be the number of black nodes in property 5. Let the shortest possible path from the root of T to any leaf consist of B black nodes. Longer possible paths may be constructed by inserting red nodes. However, property 4 makes it impossible to insert more than one consecutive red node. Therefore, ignoring any black NIL leaves, the longest possible path consists of $2 \times B$ nodes, alternating black and red (this is the worst case). Counting the black NIL leaves, the longest possible path consists of $2 \times B - 1$ nodes.

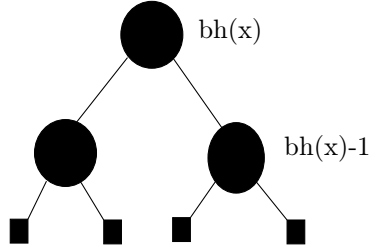
The shortest possible path has all black nodes, and the longest possible path alternates between red and black nodes. Since all maximal paths have the same number of black nodes, by property 5, this shows that no path is more than twice as long as any other path.

4 How Red Black Tree is Height Balanced

Red Black Tree for storing n items will have a height of $\mathcal{O}(n)[2]$. So, it will remain height balanced.

1. the subtree rooted at any node x contains at least $2^{bh(x)} - 1$ internal nodes.
2. If $h(x) = 0$ then x is a leaf, so the subtree rooted at x contains $2^0 - 1 = 0$ internal nodes.
3. For a node x with positive height and two children, the black height of the children will be $bh(x)$ (for red internal node) or $bh(x) - 1$ (for black internal node).





So, using induction, we can prove subtree rooted at x contains at least

- $(2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 = (2^{bh(x)} - 1)$ internal nodes.
- According to the internal property the black-height of the root must be at least $\frac{h}{2}$; thus
- $n \geq 2^{\frac{h}{2}} - 1$
- $\log(n+1) \geq \frac{h}{2}$
- $h \geq 2\log(n+1)$
- Thus $h = \mathcal{O}(\log(n))$

5 Operations

As discussed in Section 1, Red Black Trees can perform dynamic-set operations like other binary search trees. But the main advantage with Red Black Trees is they have balanced height and so those operations can be done in $\mathcal{O}(\log n)$ time in every case.

5.1 Searching

Searching in a Red Black Tree is not different than any other Binary Search Tree. We start searching from the root and compare every node with our desired key. While searching, if the required key is greater than the key of the node we are comparing with, we will go to the right child, and if it is less we will go to the left child, and repeat the same. If we reach a leaf node in this way, we can decide that the key is not in there in the tree.

The algorithm to search[3] a key in a binary tree is given below. Here we will start from the root of the tree. Input x is initially root and input k is the desired key.

```

RBT_Search(x,k)
if  $x == nil$  or  $k == x.key$  then
|   return  $x$ 
if  $k < x.key$  then
|   return RBT_Search(x.left,k)
else
|   return RBT_Search(x.right,k)

```

An example of searching 22 is simulated below in Figures 3 to 6:

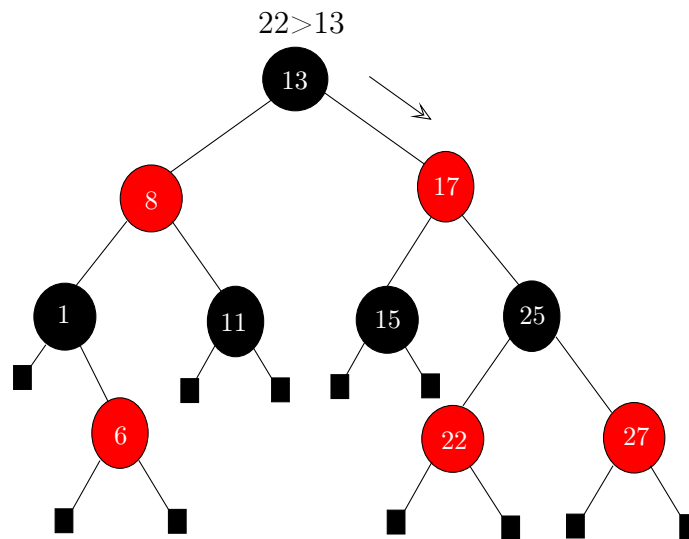


Figure 3: Searching 22 : Step 1

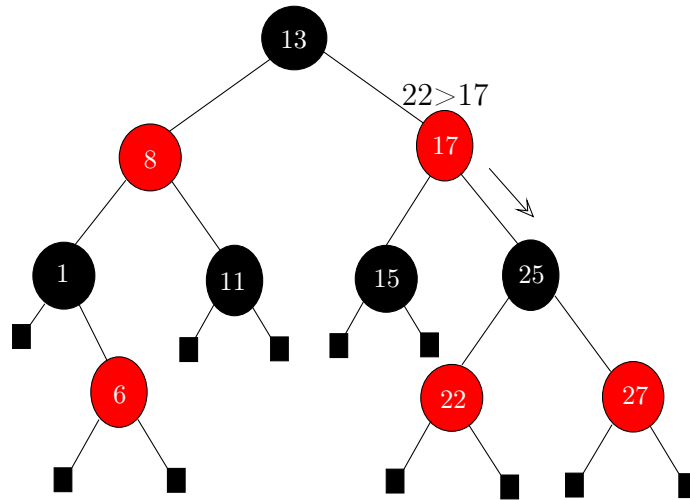


Figure 4: Seaching 22 : Step 2

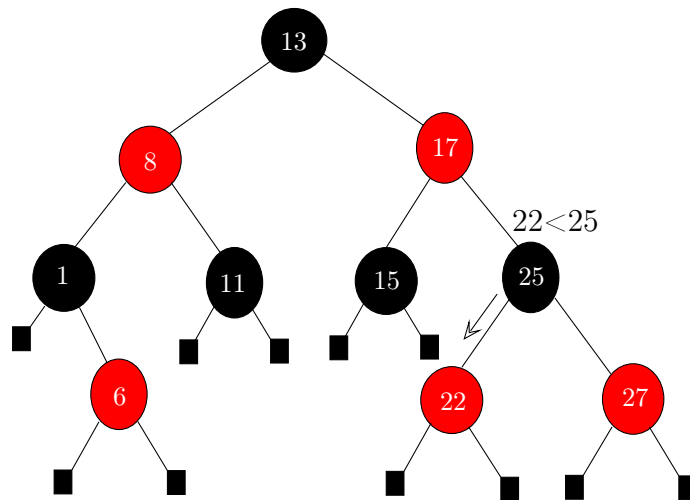


Figure 5: Seaching 22 : Step 3

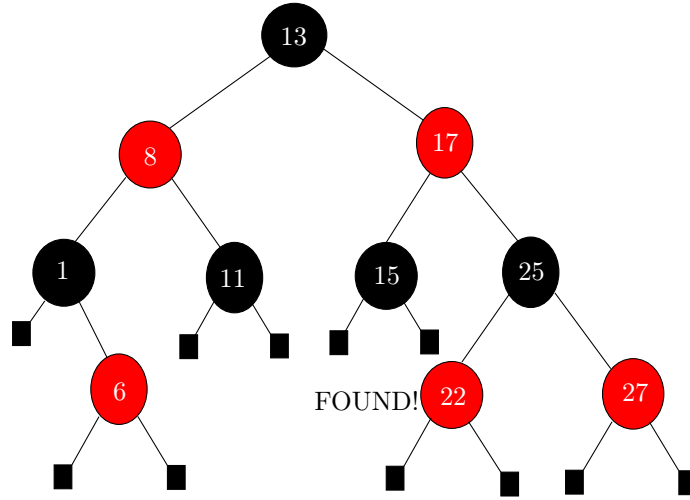


Figure 6: Searching 22 : Step 4

5.2 Insertion

Insertion begins by adding the node in a very similar manner as a standard binary search tree insertion and by coloring it red. The big difference is that in the binary search tree a new node is added as a leaf, whereas leaves contain no information in the red-black tree, so instead the new node replaces an existing leaf and then has two black leaves of its own added.

What happens next depends on the color of other nearby nodes. There are several cases of red-black tree insertion to handle:

1. New Node (**N**) is the root node, i.e., first node of red-black tree
2. **N**'s parent (**P**) is black
3. **P** is red (so it can't be the root of the tree) and **N**'s uncle (**U**) is red
4. **P** is red and **U** is black

Case 1: The current node **N** is at the root of the tree. In this case, it is repainted black to satisfy property 2 (the root is black). Since this adds one black node to every path at once, property 5 (all paths from any given node to its leaf nodes contain the same number of black nodes) is not violated.

Case 2: The current node's parent **P** is black, so property 4 (both children of every red node are black) is not invalidated. In this case, the tree is still valid. Property 5 (all paths from any given node to its leaf nodes contain the same number of black nodes) is not threatened, because the current node **N** has two black leaf children, but because **N** is red, the paths

through each of its children have the same number of black nodes as the path through the leaf it replaced, which was black, and so this property remains satisfied.

Case 3: If both the parent **P** and the uncle **U** are red, then both of them can be repainted black and the grandparent **G** becomes red to maintain property 5 (all paths from any given node to its leaf nodes contain the same number of black nodes). Since any path through the parent or uncle must pass through the grandparent, the number of black nodes on these paths has not changed. However, the grandparent **G** may now violate Property 2 (The root is black) if it is the root or Property 4 (Both children of every red node are black) if it has a red parent. To fix this, the tree's red-black repair procedure is rerun on **G**.

Note that this is a tail-recursive call, so it could be rewritten as a loop. Since this is the only loop, and any rotations occur after this loop, this proves that a constant number of rotations occur.

Case 4, step 1: The parent **P** is red but the uncle **U** is black. The ultimate goal will be to rotate the current node into the grandparent position, but this will not work if the current node is on the "inside" of the subtree under **G** (i.e., if **N** is the left child of the right child of the grandparent or the right child of the left child of the grandparent). In this case, a left rotation on **P** that switches the roles of the current node **N** and its parent **P** can be performed. The rotation causes some paths (those in the subtree labeled "1") to pass through the node **N** where they did not before. It also causes some paths (those in the subtree labeled "3") not to pass through the node **P** where they did before. However, both of these nodes are red, so property 5 (all paths from any given node to its leaf nodes contain the same number of black nodes) is not violated by the rotation. After this step has been completed, property 4 (both children of every red node are black) is still violated, but now we can resolve this by continuing to step 2.

Case 4, step 2: The current node **N** is now certain to be on the "outside" of the subtree under **G** (left of left child or right of right child). In this case, a right rotation on **G** is performed; the result is a tree where the former parent **P** is now the parent of both the current node **N** and the former grandparent **G**. **G** is known to be black, since its former child **P** could not have been red without violating property 4. Once the colors of **P** and **G** are switched, the resulting tree satisfies property 4 (both children of every red node are black). Property 5 (all paths from any given node to its leaf nodes contain the same number of black nodes) also remains satisfied, since all paths that went through any of these three nodes went through **G** before, and now they all go through **P**.

In the algorithm[3] above, all cases are called only once, except in Case 3 where it can recurse back to Case 1 with the grandparent node, which is the only case

where an iterative implementation will effectively loop. Because the problem of repair in that case is escalated two levels higher each time, it takes maximally $\frac{h}{2}$ iterations to repair the tree (where h is the height of the tree). Because the probability for escalation decreases exponentially with each iteration the average insertion cost is practically constant.

All these cases can be handles in the algorithm[3] stated below:

```

RB Insert(T,z)
   $y \leftarrow T.nil$ 
   $x \leftarrow T.root$ 
  while  $x \neq T.nil$  do
     $y \leftarrow x$ ;
    if  $z.key < x.key$  then
      |  $x \leftarrow x.left$ 
    else
      |  $x \leftarrow x.right$ 
    end
  end
   $z.p \leftarrow y$ 
  if  $y = T.nil$  then
    |  $T.root \leftarrow z$ 
  else if  $z.key < y.key$  then
    |  $y.left \leftarrow z$ 
  else
    |  $y.right \leftarrow z$ 
   $z.left \leftarrow T.nil$ 
   $z.right \leftarrow T.nil$ 
   $y.color \leftarrow red$ 
  RB-Insert-Fixup(T, z)

```

and the RB_Insert_Fixup is:

```

RB_Insert_Fixup(T,z)
while (z.parent ≠ nil) and (z.parent.color = red) do
  if z.parent = z.parent.parent.left then
    y ← z.parent.parent.right if y.color = red then
      z.parent.color ← black // CASE 1
      y.color ← black
      z.parent.parent.color ← red
      z ← z.parent.parent
    else
      if z = z.parent.right then
        z ← z.parent // CASE 2
        Left_Rotate(T, z)
      z.parent.color ← black // CASE 3
      z.parent.parent.color ← red
      Right_Rotate(T, z.parent.parent)
    else
      y ← z.parent.parent.left if y.color = red then
        z.parent.color ← black // CASE 4
        y.color ← black
        z.parent.parent.color ← red
        z ← z.parent.parent
      else
        if z = z.parent.left then
          z ← z.parent // CASE 5
          Right_Rotate(T, z)
        z.parent.color ← black // CASE 6
        z.parent.parent.color ← red
        Left_Rotate(T, z.parent.parent)
      end
    end
  end
end
root.color ← black

```

To construct a Red Black Tree consecutive insertions are performed. An example of inserting keys 2,3,5,7,4 consecutively into an initially empty tree is given below in Figures 7 to 11:

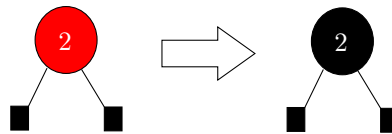


Figure 7: Inserting 2

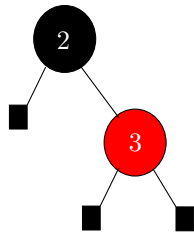


Figure 8: Inserting 3

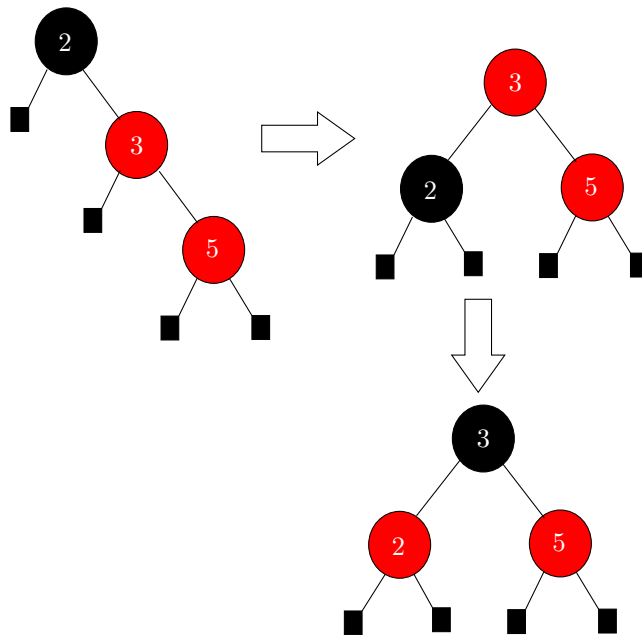


Figure 9: Inserting 5

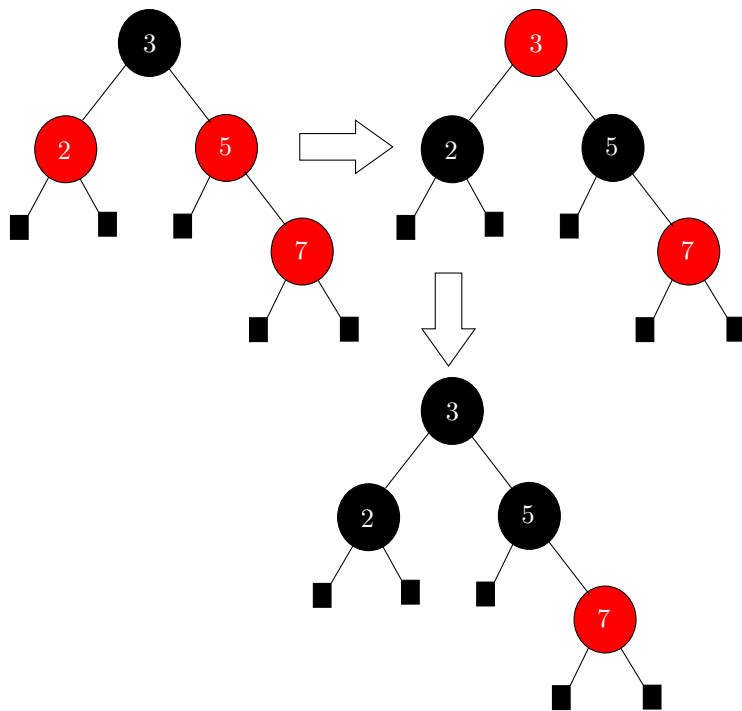


Figure 10: Inserting 7

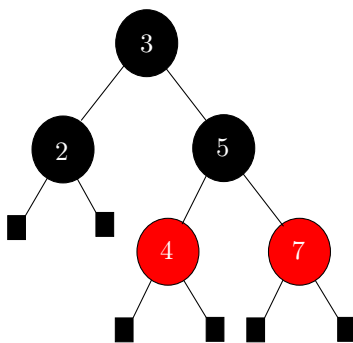


Figure 11: Inserting 4

5.3 Deletion

Like the other basic operations on an n -node red-black tree, deletion of a node takes time $\mathcal{O}(n)$. Deleting a node from a red-black tree is a bit more complicated than inserting a node.

In order to simplify boundary conditions in the code, we use a sentinel to represent **NIL**. For a red-black tree T , the sentinel $nil[T]$ is an object with the same fields as an ordinary node in the tree. Its color field is BLACK, and its other fields— p , left, right, and key—can be set to arbitrary values. In the red-black tree, all pointers to **NIL** are replaced by pointers to the sentinel $nil[T]$.

We use sentinels so that we can treat a **NIL** child of a node x as an ordinary node whose parent is x . We could add a distinct sentinel node for each **NIL** in the tree, so that the parent of each **NIL** is well defined, but that would waste space. Instead, we use the one sentinel $nil[T]$ to represent all the **NIL**'S. When we wish to manipulate a child of a node x , however, we must be careful to set $p[nil[T]]$ to x first.

The procedure RB-DELETE[3] is a minor modification of the TREE-DELETE procedure. After splicing out a node, it calls an auxiliary procedure **RB-DELETE-FIXUP** that changes colors and performs rotations to restore the red-black properties.

```

RB Delete( $T, z$ )
  if  $left[z] = nil[T]$  or  $right[z] = nil[T]$  then
    |  $y \leftarrow z$ 
  else
    |  $y \leftarrow TREE - SUCCESSOR(z)$ 

  if  $left[y] \neq nil[T]$  then
    |  $x \leftarrow left[y]$ 
  else
    |  $x \leftarrow right[y]$ 

   $p[x] \leftarrow p[y]$ 

  if  $p[y] = nil[T]$  then
    |  $root[T] \leftarrow x$ 
  else if  $left[p[y]] \leftarrow x$  then
    |  $left[p[y]] \leftarrow x$ 
  else
    |  $right[p[y]] \leftarrow x$ 

  if  $y \neq z$  then
    |  $key[z] \leftarrow key[y]$ 

  if  $color[y] = BLACK$  then
    | RB-Insert-Fixup( $T, z$ )

  return  $y$ 

```

We can now examine how the procedure RB-DELETE-FIXUP restores the red-black properties to the search tree.

```

RB_Delete_Fixup(T,z)
while ( $x \neq \text{root}[T]$ ) and ( $\text{color}[x] = \text{BLACK}$ ) do
    if  $x = \text{left}[p[x]]$  then
         $w \leftarrow \text{right}[p[x]]$ 
        if  $\text{color}[w] \leftarrow \text{RED}$  then
             $\text{color}[w] \leftarrow \text{BLACK}$ 
             $\text{color}[p[x]] \leftarrow \text{RED}$ 
            LEFT-ROTATE(T, P[X])
             $w \leftarrow \text{right}[p[x]]$ 
        if  $\text{color}[\text{left}[w]] = \text{BLACK}$  and  $\text{color}[\text{right}[w]] = \text{BLACK}$  then
             $\text{color}[w] \leftarrow \text{RED}$ 
             $x \leftarrow p[x]$ 
        else if  $\text{color}[\text{right}[w]] = \text{BLACK}$  then
             $\text{color}[\text{left}[w]] \leftarrow \text{BLACK}$ 
             $\text{color}[w] \leftarrow \text{RED}$ 
            RIGHT-ROTATE(T, P[X])
             $w \leftarrow \text{right}[p[x]]$ 
         $\text{color}[w] \leftarrow \text{color}[p[x]]$ 
         $\text{color}[p[x]] \leftarrow \text{BLACK}$ 
         $\text{color}[\text{right}[w]] \leftarrow \text{BLACK}$ 
        LEFT-ROTATE(T, P[X])
         $x \leftarrow \text{root}[T]$ 
    else
        Same as then clause with right and left exchanged
    end
 $\text{color}[x] \leftarrow \text{BLACK}$ 

```

There are several cases of red-black tree deletion to handle:

Case 1: This occurs when node w , the sibling of node x , is red. Since w must have black children, we can switch the colors of w and $p[x]$ and then perform a left-rotation on $p[x]$ without violating any of the red-black properties. The new sibling of x , one of w 's children, is now black, and thus we have converted case 1 into case 2, 3, or 4.

Case 2: In case 2 both of w 's children are black. Since w is also black, we take one black off both x and w , leaving x with only one black and leaving w red, and add an extra black to $p[x]$. We then repeat the while loop with $p[x]$ as the new node x . Observe that if we enter case 2 through case 1, the color c of the new node x is red, since the original $p[x]$ was red, and thus the loop terminates when it tests the loop condition.

Case 3: This case occurs when w is black, its left child is red, and its right child is black. We can switch the colors of w and its left child $left[w]$ and then perform a right rotation on w without violating any of the red-black properties. The new sibling w of x is now a black node with a red right child, and thus we have transformed case 3 into case 4.

Case 4: This case occurs when node x 's sibling w is black and w 's right child is red. By making some color changes and performing a left rotation on $p[x]$, we can remove the extra black on x without violating any of the red-black properties. Setting x to be the root causes the while loop to terminate when it tests the loop condition.

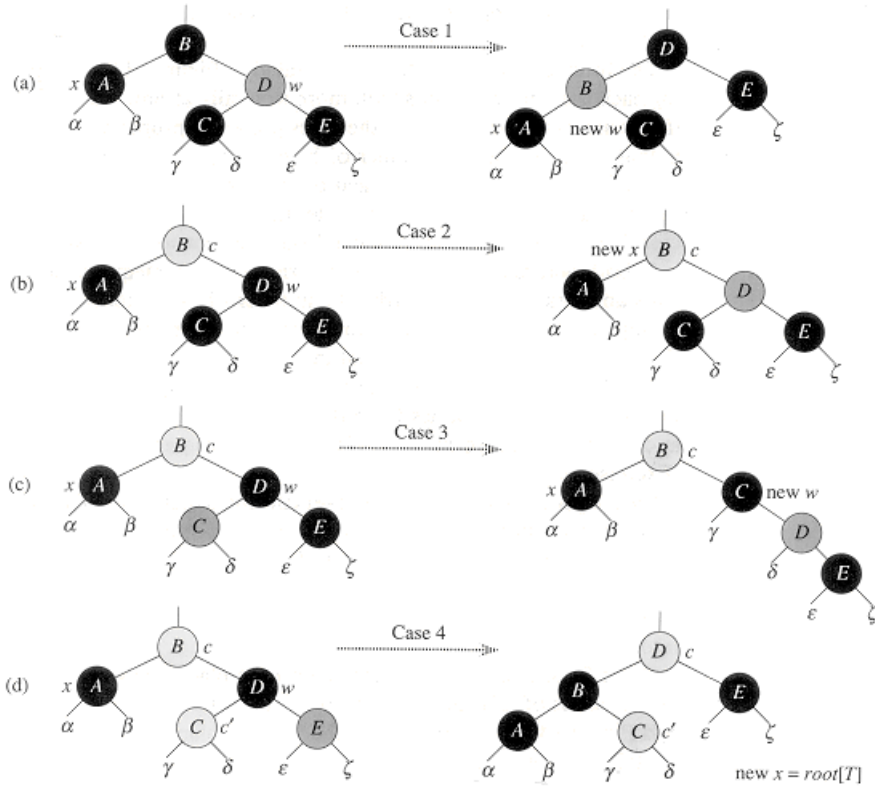


Figure 12: Red-Black Tree Deletion

6 Practical Applications

Red-black trees have numerous practical applications[1] across various domains due to their efficient balancing properties and optimized operations. Let's drive

into some detailed practical applications of red-black trees:

- Red-Black Trees are crucial for organizing files efficiently in file systems.
- Red-Black Trees enable quick retrieval of records in databases based on keys.
- Red-Black Trees are used to implement symbol tables for quick lookup of identifiers in compilers.

7 Conclusion

Balanced search trees have a height that is always $\mathcal{O}(\log n)$. One consequence of this is that lookup, insert, and delete on a balanced search tree can be done in $\mathcal{O}(\log n)$ worst-case time. In contrast, binary search trees have a worst-case height of $\mathcal{O}(n)$ and lookup, insert, and delete are $\mathcal{O}(n)$ in the worst-case. Red-black trees are just one example of a balanced search tree.

Red-black trees are binary search trees that store one additional piece of information in each node (the node's color) and satisfy three properties[4] These properties deal with the way nodes can be colored (the root property and the red property) and the number of black nodes along paths from the root node to a null child pointer (the black property). Although it is not intuitively obvious, the algorithms for restoring these properties after a node has been added or removed result in a tree that stays balanced.

Red-black trees offer efficient insertion, deletion and lookup operations, all with a worst-case time complexity of $\mathcal{O}(n)$. This is a significant improvement over binary search trees, where these operations can degrade to $\mathcal{O}(n)$ in the worst case. The algorithms for insertion and deletion in red-black trees ensure that the tree maintains its balance while satisfying the properties of red-black trees. During insertion, if the properties are violated, the tree is re-balanced through rotations and recoloring operations, ensuring that the height of the tree remains logarithmic. Similarly, deletion operations are carefully managed to maintain the balance and properties of the tree. As a result, red-black trees provide consistent and efficient performance for dynamic data structures, making them suitable for a wide range of applications.

Lookup operations in red-black trees are also optimized for efficiency. The properties of red-black trees guarantee that the path from the root to any leaf has at most twice as many black nodes as any other path, ensuring balanced access times for all elements in the tree. This balanced access pattern, combined with the logarithmic height of the tree, ensures that lookup operations are fast and predictable, even in the worst-case scenarios.

Red-black trees are not only important theoretically but also have significant practical implications in computer science. Their efficient balancing properties make them ideal for various applications where dynamic sets or maps are required. For instance, they are widely used in the implementation of associative arrays, symbol tables, and in the internals of many popular data structures and

algorithms, such as Java's[4] Tree-Map and Tree-Set. Moreover, the $\mathcal{O}(\log n)$ worst-case time complexity of operations like lookup, insert, and delete ensures predictable performance, making red-black trees suitable for real-time systems and applications where performance is critical. Additionally, their self-balancing nature reduces the chances of worst-case scenarios, providing a reliable and consistent performance guarantee even in dynamic environments.

Furthermore, red-black trees serve as a foundational concept in understanding more advanced balanced search trees and algorithms. Knowledge of their properties and operations provides a solid basis for exploring more complex data structures and algorithms, such as AVL trees, B-trees, and various self-adjusting binary search trees.

In conclusion, the study and understanding of red-black trees not only contribute to the development of efficient data structures and algorithms but also play a vital role in enhancing the performance and reliability of software systems across various domains.

References

- [1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data structures and algorithms in Python*. Wiley Publishing, 2013.
- [2] R. Sedgewick and K. Wayne. *Algorithms*. Addison-wesley professional, 2011.
- [3] C. Thomas H, E. Charles, R. Ronald L, S. Clifford, et al. *Introduction to Algorithms Third Edition*. MIT Press, 2009.
- [4] M. A. Weiss et al. *Data structures and algorithm analysis in Java*. Pearson Addison Wesley, 2007.